VARAXIS · PROTOTYPE DOCUMENTATION

ClinicOS

A local-first consultation-documentation assistant for single-doctor clinics. Architecture, workflow, and dependency reference for the working prototype.

Status Working prototype — consultation flow, records, crash recovery and backup built and tested

Repository realishaanseal/clinicos · commit 7851455

Stack Next.js 15 · React 19 · TypeScript · SQLite · faster-whisper · Ollama

Field site Krishnanagar, Nadia, West Bengal

Date September 2026

01 What the prototype does

A doctor in a high-volume clinic gets roughly two minutes per patient. In that time they must greet, take a history, examine, explain, prescribe *and* record. The record is what gets dropped.

ClinicOS records the consultation that was already happening, transcribes it on the clinic's own machine, drafts a structured clinical note from the transcript, and asks the doctor only to review and sign. A branded prescription prints from the signed note.

Every stage runs on local hardware. There is no cloud service, no account, and no internet requirement at run time.

> **Design boundary.** The system stores, formats and communicates clinical data. It does not generate diagnoses, risk scores or triage flags. This keeps it outside medical-device regulation under India's CDSCO rules, and it is enforced in the model's system prompt rather than left to convention.

02 Consultation workflow

The interface presents four steps — **Consent → Record → Transcript → Review & sign** — over a six-stage data lifecycle.

STAGE 1
Consent
Doctor reads the consent script aloud. Recording endpoints return HTTP 403 until `consentGiven` is set — the gate is server-side, not UI-only.

→

STAGE 2
Record
Browser captures audio and streams it to disk in 5-second WebM chunks, so a crash costs at most five seconds rather than the whole consultation.

→

STAGE 3
Transcribe
Chunks are finalised and posted to the local faster-whisper sidecar. Language auto-detects, which handles Bengali–English code-mixing better than forcing a locale.

STAGE 4
Draft
Transcript goes to a local Ollama model, which returns structured JSON matching the `ClinicalNote` shape. Stored as `draftJson`, immutable from this point.

→

STAGE 5
Review & sign
Doctor edits into `workingJson`, autosaved to server and localStorage. Signing writes `finalJson`, deletes the audio, and records each field diff in `Edit`.

→

STAGE 6
Print
An A4 prescription renders from the signed note, carrying clinic and doctor identity read from environment configuration.

Why the draft and the edits are stored separately

`draftJson` holds the model's original output and is never overwritten. Doctor corrections go to `workingJson`. The diff between the two at signing time is the accuracy metric — it is what shows whether the drafting is actually good enough, without anyone having to self-report.

Recovery behaviour

Audio is written continuously rather than at the end, and the in-progress note is mirrored to two independent places. Unfinished consultations resurface on the Today page annotated with the reason they stopped. A truncated WebM file still decodes up to the truncation point, which is what makes partial-recording recovery work.

03 Runtime architecture

Three processes run on the clinic machine. None of them accepts a connection from outside it.

:3000

Next.js application

App Router. Serves the interface and every API route. Owns the SQLite database through `better-sqlite3`.

:8756

ASR sidecar

FastAPI + uvicorn wrapping faster-whisper. Speech to text, entirely on-device.

:11434

Ollama

Runs `qwen2.5:7b-instruct` for note drafting. Installed as a native application, not bundled.

Provider abstraction

Speech recognition and note generation each sit behind a small interface — `AsrProvider` and `LlmProvider`, resolved by `getAsr()` and `getLlm()`. A hosted endpoint can replace either local one through environment variables alone, with no application code change. Components and route handlers never call Ollama or Whisper directly.

INTERFACE	LOCAL (DEFAULT)	ALTERNATIVE
`AsrProvider`	`local` — Python sidecar	`http` — any OpenAI-compatible `/audio/transcriptions` endpoint
`LlmProvider`	`ollama` — on-device	`openai` — any OpenAI-compatible chat-completions endpoint

Storage

SQLite accessed through `better-sqlite3` with hand-written SQL. **There is no ORM** — Prisma was removed early because its engine downloads were blocked, and it is not being reintroduced. All schema, migrations and queries live in a single module, `src/lib/db.ts`.

SQLite has no `ADD COLUMN IF NOT EXISTS`, so migrations read `PRAGMA table_info` and conditionally `ALTER TABLE`. The operation is idempotent and safe to re-run, because existing pilot databases must survive upgrades rather than being dropped and recreated.

> **Backups use SQLite's online backup API, never a file copy.** In WAL mode, recent writes live in a separate `-wal` file. A plain `cp` produces a database that opens without error but is silently missing the most recent consultations — demonstrated here with a 4 KB main file and a 453 KB WAL, which restored to "no such table: Patient".

04 Data model

Three tables. Consultation content is sensitive personal data under India's Digital Personal Data Protection Act, 2023; the database, its WAL sidecars, backups and audio are all excluded from version control.

Patient

COLUMN	TYPE	NOTES
`id`	`TEXT`	Primary key
`name`	`TEXT`	Not null; indexed for search
`age`	`INTEGER`	
`sex`	`TEXT`	
`phone`	`TEXT`	
`guardianName`	`TEXT`	
`createdAt`	`TEXT`	Not null

Consult

COLUMN	TYPE	NOTES
`id`	`TEXT`	Primary key
`patientId`	`TEXT`	FK → Patient, `ON DELETE CASCADE`; indexed
`consentGiven`	`INTEGER`	Default 0. Recording endpoints 403 while false
`consentAt / consentBy`	`TEXT`	When, and who took it
`audioPath`	`TEXT`	Cleared when audio is deleted at signing
`audioBytes / audioSeconds`	`INT / REAL`	Retained as metrics after deletion
`audioDeletedAt`	`TEXT`	Proof the deletion promise was kept
`recordingStartedAt`	`TEXT`	Migration-added
`recordingEndedAt`	`TEXT`	Migration-added; null while a recording is open
`transcript`	`TEXT`	ASR output
`asrModel / asrSeconds`	`TEXT / REAL`	Which model, and how long it took
`draftJson`	`TEXT`	**Immutable once written.** The model's original note
`workingJson`	`TEXT`	Migration-added. Doctor's in-progress edits, autosaved
`workingSavedAt`	`TEXT`	Migration-added
`finalJson`	`TEXT`	Written at signing
`llmModel / llmSeconds`	`TEXT / REAL`	
`signedAt`	`TEXT`	
`createdAt`	`TEXT`	Not null; indexed

Edit

Every correction the doctor makes to the AI draft — the accuracy metric and the training signal. Never bypassed.

COLUMN	TYPE	NOTES
`id`	TEXT	Primary key
`consultId`	TEXT	FK → Consult, `ON DELETE CASCADE`; indexed
`field`	TEXT	Which note field changed
`before / after`	TEXT	Both not null
`createdAt`	TEXT	Not null

ClinicalNote — the structured note shape

The LLM returns JSON parsed into this type. `parseNote()` is defensive: malformed or partial output degrades to an empty note rather than throwing.

FIELD	TYPE
`chiefComplaint, history, examination, diagnosis`	string
`medications`	Medication[] — name, dose, frequency, duration, instructions
`investigations`	string[]
`advice, followUp`	string

05 Dependencies — clinical application

The clinical app is a single Next.js project: the frontend and the API routes ship together, so one `package.json` covers both tiers. The Python sidecar is a separate process with its own requirements.

FRONTEND BACKEND BOTH TIERS BUILD-TIME

Node runtime dependencies

PACKAGE	VERSION	TIER	PURPOSE
`next`	`^15.1.0`	BOTH	App Router framework. Serves the UI and hosts every API route handler
`react`	`^19.0.0`	FE	UI runtime
`react-dom`	`^19.0.0`	FE	DOM renderer
`better-sqlite3`	`^13.0.3`	BE	Synchronous SQLite driver. Supplies the online backup API. **Must stay on v13+** — see note below
`motion`	`^13.2.0`	FE	Animation primitives (successor to Framer Motion)
`gsap`	`^3.15.0`	FE	Timeline animation and ScrollTrigger
`lenis`	`^1.3.26`	FE	Smooth-scroll driver, run off the GSAP ticker
`three`	`^0.186.0`	FE	WebGL renderer
`@react-three/fiber`	`^9.7.0`	FE	React reconciler for three.js
`@react-three/drei`	`^10.7.8`	FE	Helper components for R3F

> **Motion and 3D are scoped, not global.** Clinical surfaces get functional motion under 200 ms and no 3D at all; expressive motion and WebGL belong only on marketing surfaces, loaded behind `next/dynamic` with `ssr: false`. A global `prefers-reduced-motion` guard is a requirement rather than a nicety — this is health software, and motion sickness in a clinic is not cosmetic. The recording dot and waveform degrade to a static visible state rather than freezing, because their meaning is carried by movement.

Build-time dependencies

PACKAGE	VERSION	PURPOSE
`typescript`	`^5.7.0`	Language and type checking
`@types/node`	`^22.10.0`	Node type definitions
`@types/react`	`^19.0.0`	React type definitions
`@types/react-dom`	`^19.0.0`	React DOM type definitions
`@types/better-sqlite3`	`^7.6.13`	SQLite driver type definitions
`@types/three`	`^0.185.4`	three.js type definitions

Python sidecar — `asr/requirements.txt`

PACKAGE	VERSION	PURPOSE

`faster-whisper`	`≥1.1.0`	CTranslate2 Whisper inference. Does the actual transcription
`fastapi`	`≥0.115`	HTTP interface for the sidecar
`uvicorn`	`≥0.32`	ASGI server
`python-multipart`	`≥0.0.12`	Multipart parsing for audio upload

External runtime prerequisites

Not installed by the package managers — these are native applications the setup script locates rather than bundles.

COMPONENT	REQUIREMENT
`Node.js`	LTS. `winget install OpenJS.NodeJS.LTS` on Windows
`Python 3`	Virtualenv created by the setup script for the sidecar
`Ollama`	Native app, then `ollama pull qwen2.5:7b-instruct`

> **Two install traps, both already paid for.**
> **npm 12 blocks install scripts by default**, so `better-sqlite3`'s native binary silently fails to install, `npm install` reports success, and the app crashes on first database access. `package.json` carries an `allowScripts` allowlist (`better-sqlite3`, `sharp`) and the launcher verifies the module actually loads.
>
> **`better-sqlite3` must stay on v13 or newer.** v11 shipped ABI-specific prebuilds, so a newer Node version had none and npm fell back to compiling from source — which needs Visual Studio on Windows and Xcode tools on macOS. v13 moved to N-API: one binary per platform, nothing compiles. Do not pin it back.

06 API surface

Sixteen route handlers under `src/app/api`. All are local-only; none is exposed beyond the machine.

Consultation lifecycle

ROUTE
`/api/consults`
`/api/consults/[id]/consent`
`/api/consults/[id]/audio`
`/api/consults/[id]/audio/chunk`
`/api/consults/[id]/audio/finalize`
`/api/consults/[id]/transcribe`
`/api/consults/[id]/draft`
`/api/consults/[id]/note`
`/api/consults/[id]/sign`

Patients

ROUTE
`/api/patients`
`/api/patients/[id]`
`/api/patients/[id]/export`

Backup & export

ROUTE
`/api/backup/create`
`/api/backup/download`
`/api/backup/restore`
`/api/backup/export`

Backup and restore behaviour

A snapshot is taken lazily on page load, once daily, and fourteen are kept. Download is one tap. Restore sits behind a typed `RESTORE` confirmation and snapshots the current database before replacing it. Full export in an open format is a standing requirement — no lock-in.

07 Source layout

PATH	RESPONSIBILITY
`src/lib/db.ts`	All storage — schema, migrations, every query. No ORM
`src/lib/backup.ts`	Snapshot, restore, retention
`src/lib/asr.ts`	Speech-to-text provider interface (local | http)
`src/lib/llm.ts`	Note-generation provider interface (ollama | openai). Holds the clinical safety system prompt
`src/lib/types.ts`	`ClinicalNote` shape and `parseNote()`
`src/lib/clinic.ts`	Clinic identity read from environment
`src/lib/useRecorder.ts`	Streams 5-second audio chunks to disk while recording
`src/lib/useAutosave.ts`	Debounced server save plus localStorage mirror
`src/components/ConsultWorkflow.tsx`	The four-step consultation UI
`src/components/Prescription.tsx`	Printable A4 prescription
`src/components/BackupPanel.tsx`	Settings: backup, restore, export

`src/components/DeletePatient.tsx`	Type-the-name-to-confirm deletion
`asr/server.py`	FastAPI + faster-whisper sidecar
`go.sh / go.ps1`	Idempotent setup and launch, macOS and Windows

08 Configuration

Copy `.env.example` to `.env`. No secret is required for the default fully-local configuration.

VARIABLE	DEFAULT	PURPOSE
DB_PATH	./data/clinicos.db	SQLite file location
AUDIO_DIR	./data/audio	Where audio is written before deletion at signing
ASR_PROVIDER	local	`local` keeps audio on the machine; `http` uses a remote endpoint
ASR_LOCAL_URL	127.0.0.1:8756	Sidecar address
ASR_LANGUAGE	(empty)	Empty = auto-detect. Handles code-mixing better than forcing `bn`
ASR_HTTP_URL / KEY / MODEL	–	Only for the `http` provider
LLM_PROVIDER	ollama	`ollama` is fully local; `openai` uses a remote endpoint
LLM_OLLAMA_URL	127.0.0.1:11434	Ollama address
LLM_OLLAMA_MODEL	qwen2.5:7b-instruct	Drafting model
LLM_OPENAI_URL / KEY / MODEL	–	Only for the `openai` provider
CLINIC_NAME / ADDRESS / PHONE	Demo Clinic	Printed on every prescription
DOCTOR_NAME / QUALIFICATION / REGNO	Dr. Demo	Printed on every prescription

Running it

macOS — `cd clinicos-app && ./go.sh`. **Windows** — `powershell -ExecutionPolicy Bypass -File .\go.ps1`. Both are idempotent: they install what is missing, rebuild the native SQLite binary if it did not compile, create the Python virtualenv, locate Ollama, and start all three processes. The two launchers are kept deliberately in step, because the field laptop and the development laptop are not the same operating system.

09 Landing site — a separate application

The marketing site lives at `landing/` as its own Next.js app. **It has no database, no API routes, and no access to anything in `clinicos-app`.** That separation is deliberate and is what makes it safe to deploy publicly: the clinical app must never be hosted, because patient data never leaves the clinic machine. The landing site can be, because it holds nothing.

PACKAGE	VERSION	PURPOSE
next	^15.1.0	Static site generation
react / react-dom	^19.0.0	UI runtime
gsap	^3.15.0	Scroll-linked scene morphing, MorphSVG
motion	^13.2.0	Component-level animation
lenis	^1.3.26	Smooth scroll, driven off the GSAP ticker so the two clocks agree
typescript + @types/*	^5.7.0	Build-time only

Deployment. Vercel, with Root Directory set to `landing`. No environment variables, no database, no build overrides — the page is fully static. It runs on port 3001 in development so it can sit alongside the clinical app on 3000.

The product-tour screens are rendered mock-ups of the real application's structure, not screenshots, and the page says so. An interface picture implying a running system it cannot show is the kind of thing that gets noticed in a room full of clinicians.

10 Non-negotiable invariants

Violating any of these changes what the project legally or ethically is. They are not traded away for a feature.

1. **No clinical judgement.** No diagnosis generation, no risk scores, no triage badges. Software limited to storing, formatting and communicating data sits outside medical-device regulation; software that quantifies a physiological parameter to drive a clinical decision does not. Raw numbers and trends are shown; the doctor sets any flag.
2. **Identifiable data never leaves the machine.** No cloud sync of patient data, no telemetry containing consultation content, no exceptions for testing.
3. **Audio is deleted when the doctor signs.** Promised aloud to the patient in the consent script and enforced in `signConsult()`.
4. **Never train on patient audio.** The Pilot Agreement permits de-identified data only, with an opt-out.
5. **Consent is enforced server-side.** Recording endpoints return 403 when `consentGiven` is false. A UI-only gate is not acceptable.
6. **`draftJson` is immutable once written.** Doctor edits go to `workingJson`; the diff at signing is the accuracy metric.
7. **Backups use SQLite's online backup API**, never a file copy.
8. **No lock-in.** Full export in an open format must keep working.

Interface conventions

- Plain language throughout — this is read by a doctor mid-consultation, not by a developer. "Nothing here has been lost" beats "recovery state detected".
- Anything irreversible requires typing something to confirm: a patient's name, or the word `RESTORE`.
- Deterministic rule-based behaviour is never branded as "AI".
- Numbers shown to doctors are deliberately conservative — time-saved is computed at four minutes per note, because a figure a doctor disbelieves costs more than a smaller one they accept.
- Native system font stacks, no webfonts. Google Fonts are unreachable in some environments and the dependency buys nothing.

11 Status and what is deliberately absent

Built and tested

- The full consultation flow: consent → record → transcribe → draft → sign → print
- Patient records, history, search, and type-to-confirm deletion
- Crash and offline recovery — chunked audio, dual-location autosave, unfinished consultations resurfaced with the reason they stopped
- Backup and restore — daily lazy snapshot, fourteen retained, one-tap download, guarded restore

Not built, deliberately

Hosting of the clinical application, standalone desktop packaging, multi-doctor accounts, and any AI capability beyond transcription and note drafting. Each is a decision rather than a gap.

The gate that matters. Zero of the twenty doctor discovery interviews have been done. The launch plan sets kill criteria written before any testing: by week 4, at least 6 of 20 doctors must name documentation unprompted among their top-three frustrations, or the problem is not the one assumed. The remaining backlog is roughly a year of part-time work — and building further features before those interviews is the single largest risk to the project.